LMA individual Project

Jainil Bavishi

Abstract

For this Project I have built, train and analysed two independent 'gpt-style' decoder-only transformer languague models from scratch (using only basic pytorch components). Corpus collection, tokenizer training, building model, pretraining and finetuning was all done and distributed over 3-phases which took over a month.

In this report, I will discuss all the tasks in detail, report my findings and finally give my 'take-aways' of this project. This report is completely written/typed by hand, taking some syntactic help from AI for tables and figures and such.

1 Overview

This project is implemented in 3-phases, which went over all these tasks: Corpus collection, Tokenizer training, Architecture Implementation, Pretraining, Finetuning on reasoning tasks. Each of these tasks will be its own section, within which I will delve into detail about challenges, design decisions, code implementation and key findings.

The high-resource language I have chosen is **Telugu**, and the low-resource language is **Nepalese**. Telugu is among the 4 major south indian languages of India and is spoken by 104.5 million people. In comparision Nepalese is 'only' spoken by 32 million people and more so is scarcely spoken within India (relatively).

The gh link will be provided at the end of the report for checking the codebase.

2 Corpus Collection

The guidelines for this project stipulates that we must collect around **500M** training tokens worth of corpus, out of which at least 20% of them must be procured on our own (not from already prepared corpora).

The corpus of the two LMs must be independent of each other. Machine Translation or synthetic data was not permitted. I have maintained all these markers and will report them below.

Telugu

As anticipated the major challenge in this section is going to be the manual collection of high quality Telugu text from the internet (to meet the 20% criteria). But for my convenience **wikimedia** proved to have a large library of human-written telugu articles, out of copyright novels, textbooks, etc..

Wikimedia alone satisfied the 20% criteria. The rest of corpus was supplemented by **Sangraha** which is a high quality indic corpora. I also made sure that the text from sangraha was verified and not synthetic, (sangraha provides both options).

Source	Token Count	Percentage
Sangraha	486,127,738	75.75%
Wikipedia	76,881,984	11.98%
Wikisource	70,721,157	11.02%
Wiktionary	7,315,982	1.14%
Wikibooks	705,928	0.11%
Total	**641,752,789**	**100.00%**

Table 1: Telugu Token Distribution by Source

Nepalese

Collecting corpus for Nepalese proved to be much greater challenge. My previous 'trumpcard' wikimedia proved to be less than useful

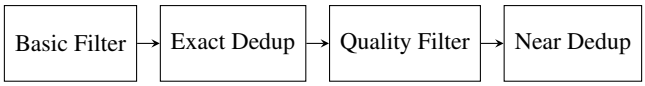


Figure 1: Data Cleaning Pipeline

this time, it only provided a fraction of the 100M tokens that I need. This meant I turned to news-websites and common-crawl. To ensure I was only extracting nepalese text and not hindi or other languages that share the same script, I used a language identification model (that was developed by meta).

Even after all that I wasn't able to make the 20% cut but I only narrowly missed. The rest of the corpus like last time was supplement by snagraha. Again I made sure to not use synthetic data.

Source	Token Count	Percentage
Sangraha	441,377,006	84.91%
CC	27,290,417	5.25%
Gorkhapatra	17,621,812	3.39%
Wikipedia	14,242,998	2.74%
Ekantipur	10,916,167	2.10%
Ratopati	8,265,098	1.59%
OnlineKhabar	103,963	0.02%
Total	**519,817,461**	**100.00%**

Table 2: Nepalese Token Distribution by Source

Data Cleaning Pipeline

For both the languages I used the same data cleaning Pipeline. This pipeline is extremely simple with only 4 stages. I could have had more thorough cleaning pipeline, but I was significantly hindered by hardware and time. Data cleaning easily was the most time-consuming and hard intensive process (in my experience). Also I felt that it wasnt really required given I was sourcing from high quality sources. Sangraha already had thoroughly cleaned text in its corpora.

1. **Basic Filter**: removes html tags, urls, control characters etc.. It also normalises whitespace, unicode.

2. **Exact Dedup**: does deduplication by storing the hashes of previously seen documents and removing the ones that has a hash-collison.

3. **Quality Filter**: checks for language ratio, sentence repetition ratio, digit ratio, lexical diversity, and removes overtly short or long documents

4. **Near Dedup**: Does near deduplication by utilising LSH+minhash.

3 Tokenizer

I chose **BPE** tokenizer, as it is the most popular tokenizer in current times. Also BPE adapts well to indian languages, so I made the choice. For its implementation I chose SentencePiece Library by Google.

I split the corpus into 99/1/1 split for train/val/test. From here on, all the training of all the modules happened on the train split only including the tokenizer.

Refer Table.3 for detailed stats for both Telugu and Nepalese Tokenizer.

4 Architecture Implementation

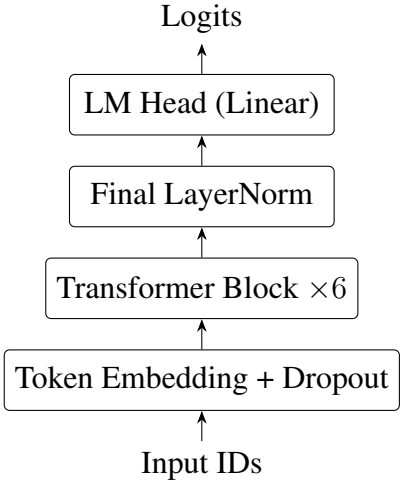


Figure 2: Overview of the Architecture of LM

Metric	Telugu	Nepalese
Total tokens in the corpus	641,752,789	519,817,461
Total words	291,052,982	310,885,940
Fertility (tokens/word)	2.20	1.67
Total unknown tokens	678,964	1,197,408
Docs with unknown tokens	12.80%	16.74%
Unknown token percentage	0.11%	0.23%
Vocabulary size	8,000	6,000
Vocabulary used	99.96%	99.95%
Average token length	4.47	4.30
Minimum token length	0	0
Maximum token length	15	15

Table 3: Tokenizer Evaluation Statistics for Telugu and Nepalese

Positional Embedding

I implemented Rotary Positional Embedding (**RoPE**). The reasons for choosing it over Learned PE or Sinusoidal PE:

- Sinusoidal PE has fixed sequence length over training time
- In RoPE there are no learnable parameters, hence making it more extensible

Essentially RoPE works by rotating Q and K matrices by doing matrix multiplications before the attention mechanism over every layer.

$$\boldsymbol{R}^{d}_{\Theta,m}\boldsymbol{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_{d-1} \\ x_d \end{pmatrix} \otimes \begin{pmatrix} \cos m\theta_1 \\ \cos m\theta_1 \\ \cos m\theta_2 \\ \cos m\theta_2 \\ \vdots \\ \cos m\theta_{d/2} \\ \cos m\theta_{d/2} \end{pmatrix} + \begin{pmatrix} -x_2 \\ x_1 \\ -x_4 \\ x_3 \\ \vdots \\ -x_d \\ x_{d-1} \end{pmatrix} \otimes \begin{pmatrix} \sin m\theta_1 \\ \sin m\theta_1 \\ \sin m\theta_2 \\ \sin m\theta_2 \\ \vdots \\ \sin m\theta_{d/2} \\ \sin m\theta_{d/2} \end{pmatrix}$$

Figure 3: How RoPE is implemented

Transformer Block

As you can see in figure.4, a transformer block consists of multi-head attention module (**MHA**), LayerNorm and Feed-Forward Network (**FFNN**) modules. Also note that this Pre-Norm architecture. We will go through each of these modules carefully.

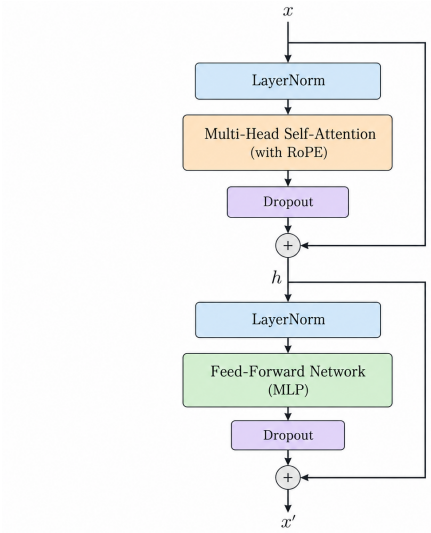


Figure 4: Transformer Block

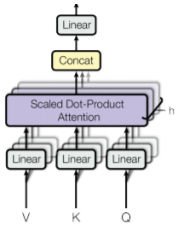


Figure 5: MHA at a glance

Language	Decoding	BLEU	chrF	ROUGE-L	Rep-2	Rep-3	Distinct-1	Distinct-2
Telugu	Greedy	1.42	11.95	0.0201	0.8260	0.8186	0.1542	0.1740
	Temp 0.5	1.82	16.15	0.0351	0.6367	0.5819	0.2782	0.3633
	Temp 1.0	0.57	23.57	0.0290	0.0468	0.0277	0.8247	0.9532
	Temp 1.5	0.18	23.11	0.0211	0.0020	0.0005	0.9536	0.9980
Nepalese	Greedy	2.50	18.00	0.0361	0.8032	0.7882	0.1720	0.1968
	Temp 0.5	1.79	20.58	0.0368	0.5409	0.4679	0.3418	0.4591
	Temp 1.0	1.04	24.82	0.0418	0.0513	0.0226	0.8084	0.9487
	Temp 1.5	0.19	22.86	0.0107	0.0034	0.0009	0.9409	0.9966

Table 4: Generation Evaluation Metrics for Telugu and Nepalese

Multi-Head Attention

Scaled Dot Product Attention is the fundamental equation for attention mechanism:

$$Attention(Q, K, V) = softmax\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{1}$$

In multi-head attention we split our Q, K, V matrices along the hidden dimension across the heads and then finally concatenate the matrices to then project it back into same dimension as we started before.

Feed-Forward NN

FFNN is a straight forward MLP which is two linear transformations with a non-linear activation in between. It is applied identically to each position.

LayerNorm

Layer Normalization is useful to stabilise the inputs and get a consisitent Loss curve.

$$LN(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \tag{2}$$

5 Pretraining

As mentioned before, pretraining occured only on training split of the dataset that has been collected. For convenience, training hyperparameters are kept same for training both LMs.

Hyperparameter	Value
Epochs	1
Learning Rate	6.0×10^{-4}
Warmup Steps	300
Min LR Ratio	0.1
Weight Decay	0.1
β_1	0.9
β_2	0.95
ϵ	1.0×10^{-8}
Grad Clip	1.0
Seed	42

Table 5: Training Hyperparameters

Results

Intrinsic metrics

I am measuring Perplexity and Bits per token.

Telugu model has a higher perplexity (38.96) compared with the Nepalese model (23.87). This indicates that the Telugu model is less confident in predicting the next token on its test set and therefore models the language less effectively under the current setup. The Telugu model also has a slightly higher bits-per-byte/token metric (0.769 vs. 0.618), further indicating higher uncertainty.

Language	Total Bytes	Perplexity	Bits per Token
Telugu	61,712,860	38.959	0.769
Nepalese	52,036,231	23.868	0.618

Table 6: Intrinsic Metrics by Language

This is a bit surprising, but I think it can be due to training/data collection. Telugu model was trained for (marginally) fewer steps that nepalese model. In data collection, for nepalese I collected text from a large collection of sources, so the diversity might have positively effected the model.

Generation Evaluation

The Nepali model generally performs better in reference-based generation metrics, while both models show a similar decoding trade-off.

Attention Evaluation

Telugu

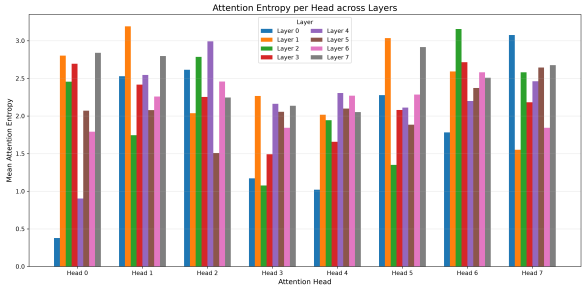


Figure 6: Attention entropy per head for Telugu Model

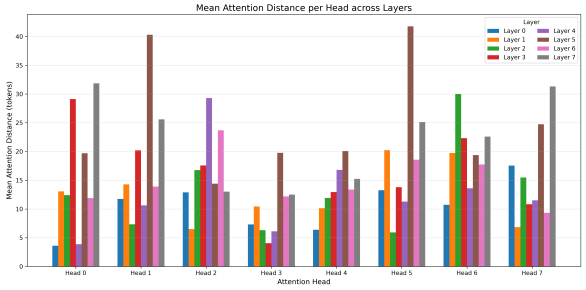


Figure 7: Mean attention entropy per head for Telugu Model

Nepalese

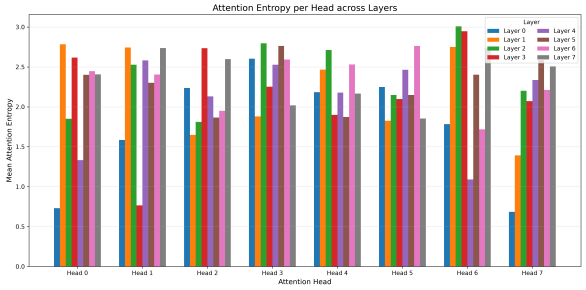


Figure 8: Attention entropy per head for Nepalese Model

The Telugu model shows substantial variation across heads, with both highly local heads and strongly long-range heads. The deeper layers generally contain more long-range heads than the earliest layers.

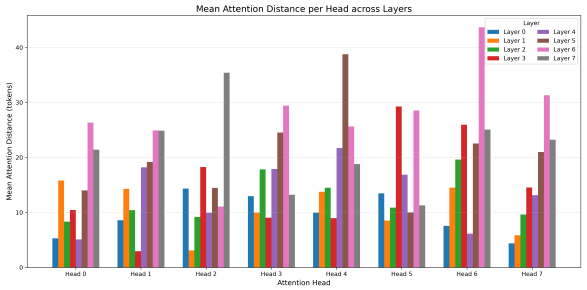


Figure 9: Mean attention entropy per head for Nepalese Model

The Nepali model shows a clear tendency for early layers to have shorter attention distances, while several deeper layers develop substantially longer-range attention. For example, Layer 0 contains heads with distances below 6 tokens, whereas Layer 6 contains heads reaching over 40 tokens. This suggests that the model may use lower layers primarily for local contextual and positional information and deeper layers for integrating more distant context

6 Finetuning

Finally, the pretrained model is now finetuned on a reasoning dataset.

Dataset Creation

I generated question->reason->answer triplet dataset by following these steps.

1. Pre-determine templates like (A>B , C>B, what is the least?). I using AI made around 20 such templates.

2. When generating the dataset, the ground truth is hardcoded, and using this and templates, dataset was generated.

Finally, 20000 train examples, 2500 validation examples and 2500 test examples are generated.

Language	Decoding	BLEU	chrF	ROUGE-L	Rep-2	Rep-3	Distinct-1	Distinct-2
Nepalese	Greedy	2.50	18.00	0.0364	0.8032	0.7882	0.1720	0.1968
	Temp 0.5	2.17	20.39	0.0361	0.5577	0.4872	0.3319	0.4423
	Temp 1.0	0.68	24.11	0.0244	0.0595	0.0309	0.7950	0.9405
	Temp 1.5	0.14	22.34	0.0085	0.0037	0.0007	0.9360	0.9963
Telugu	Greedy	1.42	11.95	0.0208	0.8260	0.8186	0.1542	0.1740
	Temp 0.5	1.62	16.57	0.0362	0.6363	0.5853	0.2837	0.3637
	Temp 1.0	1.16	24.17	0.0370	0.0425	0.0185	0.8229	0.9575
	Temp 1.5	0.19	23.13	0.0437	0.0027	0.0008	0.9538	0.9973

Table 7: Generation Evaluation Metrics for Nepalese and Telugu Finetuned Models

Hyperparameters

Hyperparameter	Value
Epochs	3
Learning Rate	2.0×10^{-5}
Grad Accum Steps	4
Batch Size	8
Weight decay	0.01
Seed	20260914

Table 8: Training Hyperparameters

Reasoning Evaluation

To evaluate reasoning, I adopted 3 metrics:

- *Strict*: Only when the generated text matches with the reference will it be deemed correct. (after normalization)
- *Loose*: If the generated text contains the reference text anywhere then it will be deemed correct.
- *Format compliance*: This is to test wether the model has learnt the format and is able to answer using the format.

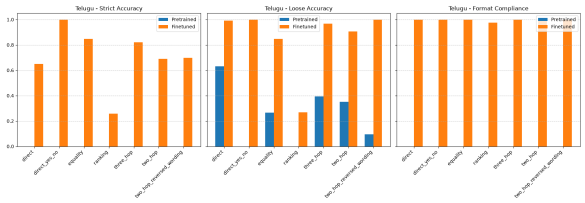


Figure 10: Reasoning evaluation for Telugu on few templates

As you can see, finetuning has improved reasoning across the board for both the language models.

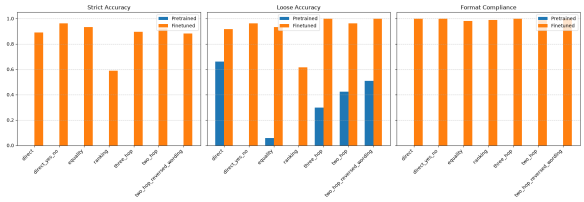


Figure 11: Reasoning evaluation for Telugu on few templates

Intrinsic Metrics

Interestingly both *finetuned and pretrained models have the exact same intrinsic metrics*. On doing more research I found the reason could be that the general language tokens that have nothing to do with reasoning finetuning are overwhelmingly in majority and hence influence the perplexity score.

Generation Evaluation

Again, the results are largely the same as pretrained one. This suggests that finetuning doesnt affect the underlying mechanism of text generation.

For formating purposes, I shifted the contents to next page....

Attention Evaluation

Telugu

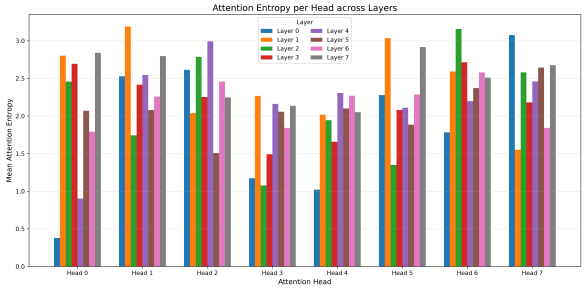


Figure 12: Attention entropy per head for Telugu Model

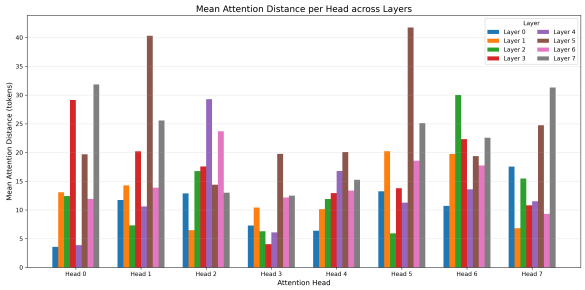


Figure 13: Mean attention entropy per head for Telugu Model

Nepalese

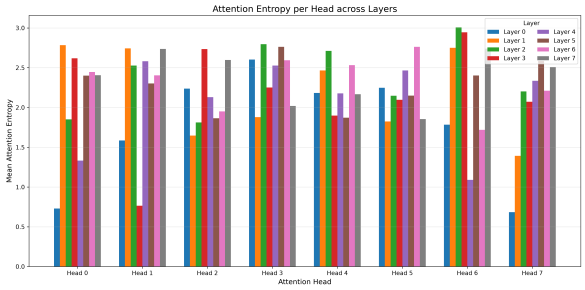


Figure 14: Attention entropy per head for Nepalese Model

Again (I might be wrong) but I dont necessarily see any changes with pretrained model.

7 Conclusion

In this project, I have learnt the design decisions one has to make when building a LM. Especially with corpus collection, I can observe the effort and understanding that goes behind it. I understood how diversity and quality data is as important if not more that the LM model architecture itself.

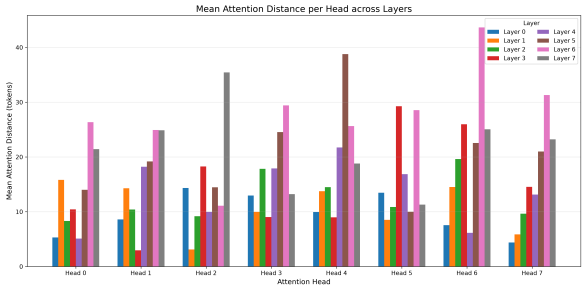


Figure 15: Mean attention entropy per head for Nepalese Model

I also learnt about the various evaluation metrics and what each metric tell about the model. Although I have been hearing about benchmarks for very long time, this is my first time actually evaluating a model.

Links

github

- PreTraining and Finetuning Dataset
- Pretrained and finetuned model checkpoints
- finetunig dataset drive link
- finetuned models drive link